

properties (road length, travel time, capacity, speed limit, vehicle density, vehicle flow, current congestion, future congestion, wave magnitude, wave direction, future wave exposure, road condition, road quality, weather impact, accident probability, road closure probability, predicted traffic, risk score). I'll need to assign consistent values. I can create tables with numeric values for each node and edge. That will be used throughout later chapters.

So I'll craft the book. I'll start with Part I.

I'll generate Mermaid diagrams: for search space explosion, maybe a graph showing number of configurations growing factorially. I'll include Mermaid flowcharts for brute force, etc. I'll be mindful of not using HTML. I'll use standard Markdown. I'll include code blocks for pseudocode when describing algorithms like Dijkstra, but that's not required for Part I. In Part II, I'll provide algorithms.

I'll make sure to explain every concept in the order: WHY, problem it solves, what happens if removed, intuition, mathematics, physical meaning, computational meaning, relation to previous/future concepts.

I'll write until I reach the response limit. I'll end with "TO BE CONTINUED IN PART ...". Let's plan the size: I need to cover a lot. I'll try to cover Parts I through XII maybe, but that's huge. I'll gauge as I write. I'll likely be able to cover Part I, II, III, IV, V, VI, VII, VIII, IX, X, XI, and maybe start XII. But I'll aim to produce a long text. I'll structure with clear sections. The response max length in this context is unknown, but I'll generate as much as I can. I'll likely exceed if I do all 16 parts thoroughly. I'll go as far as possible, then stop with "TO BE CONTINUED IN PART ...". I'll start with Part I, II, III, etc. I'll stop at a natural break, perhaps after Part XI or XII. I'll include proper Markdown. Let's begin writing.

QAOA From First Principles

*A Graduate-Level Textbook on the Quantum Approximate Optimization Algorithm
for Routing and Transportation Networks*

Part I — Optimization

What is optimization?

Optimization is the mathematical discipline of finding the *best* among a set of possible choices. The word “best” is always defined by a quantitative measure—the **objective function**—and the choices are limited by **constraints**. This simple idea underpins virtually every field of engineering, science, and decision-making. Without optimization, we could not design efficient aircraft wings, schedule factory production, route internet traffic, or plan a journey through a city.

1.1 Why Optimization Exists

We optimize because resources are finite and goals are competing. A factory wants to maximize profit but minimize waste. A delivery company wants the shortest routes but the fewest vehicles. In each case, someone must balance conflicting demands and find a solution that is, in a precise mathematical sense, optimal.

Optimization exists to answer the question:

“Given a set of possible actions, which one yields the best outcome as measured by my objective, while respecting all rules?”

Without optimization, we would rely on guesswork. For hard problems, guessing is catastrophically inefficient.

1.2 Objective Functions, Constraints, and Search Spaces

1.2.1 Objective Function

The **objective function**, $f(x)$, is the quantity we want to maximize or minimize.

Example: In a routing problem, the objective could be total travel time. The variable x encodes a route, and $f(x)$ gives the time needed to traverse it.

Why do we need it?

Without an objective function, “best” has no meaning. Every solution is equally good, and optimization collapses to mere enumeration. The objective provides a numerical yardstick against which all candidates are measured.

What happens if we remove it?

We cannot compare solutions. The problem becomes “find any feasible solution,” which is a constraint satisfaction problem, not an optimization problem. That is sometimes useful, but most real-world tasks demand optimality, not just feasibility.

Intuition:

Think of a landscape where height represents cost. Low valleys are good solutions (minimization). The objective function is the entire landscape, and we search for the lowest point.

Mathematics:

Let $\mathcal{X}$ be the set of all feasible solutions.

Minimization: $x^* = \arg \min_{x \in \mathcal{X}} f(x)$

Maximization: $x^* = \arg \max_{x \in \mathcal{X}} f(x)$

Physical meaning:

In a transportation network, $f(x)$ might sum the fuel consumed, time wasted, or pollution emitted. Minimizing it directly improves real-world efficiency.

Computational meaning:

The objective function is a subroutine that evaluates a candidate solution. Its complexity and smoothness determine which algorithms can solve the problem efficiently.

1.2.2 Constraints

Constraints restrict the allowed set $\mathcal{X}$.

Example: A vehicle cannot visit the same junction twice, or must not exceed a maximum route length.

Why constraints exist:

The real world is full of physical, legal, and logical restrictions. A solution that ignores them is useless.

What happens if we remove constraints?

The search space becomes larger but filled with invalid solutions. Algorithms may waste time exploring infeasible regions, and the final answer may be impossible to implement.

Intuition:

Constraints are like fences in the landscape—we can only walk in the fenced area.

Mathematics:

Constraints are usually expressed as equalities or inequalities:

$$g_i(x) = 0 \text{ or } h_j(x) \leq 0.$$

The set $\mathcal{X} = \{x \mid \text{all constraints satisfied}\}$.

Physical meaning:

A road has a maximum capacity. A vehicle has a limited fuel tank. A driver must obey traffic laws. All become constraints.

Computational meaning:

Constraints can be handled by penalty methods (adding a cost for violations) or by restricting the search algorithm itself. Penalty methods transform a constrained problem into an unconstrained one, often at the cost of tuning.

1.2.3 Search Space

The **search space** is the set of all possible candidate solutions.

For n binary decisions, the search space has size 2^n . For a traveling salesman on m cities, there are $(m - 1)!/2$ distinct tours. This space grows astronomically with problem size.

Why it matters:

The size and structure of the search space determine whether we can solve a problem by brute force or must resort to clever algorithms. Large discrete spaces are the heart of combinatorial optimization.

What happens if we ignore the search space?

We cannot reason about algorithm efficiency. A method that works for $n = 10$ may fail for $n = 100$ because the space is 2^{90} times larger.

Intuition:

The search space is the universe of all possible routes, schedules, or assignments. Optimization is the art of navigating this universe without visiting every star.

Computational meaning:

The search space is the domain of the objective function. Algorithms traverse it, sometimes deterministically, sometimes stochastically.

1.3 Combinatorial Optimization

Combinatorial optimization deals with problems where the variables are discrete and often finite. The optimal solution is a particular combination, permutation, or subset of items.

Why this class exists:

Many real-world decisions are discrete: you either take a road or you don't; you assign a task to worker A or worker B. Continuous optimization cannot model such choices without rounding.

Examples:

- **Traveling Salesman Problem (TSP):** Find the shortest tour visiting each city once.
- **Vehicle Routing Problem (VRP):** Design routes for a fleet to serve customers.
- **Max-Cut:** Partition graph vertices into two sets to maximize the number of crossing edges.

- **Knapsack:** Select items to maximize value without exceeding weight capacity.

These problems share a common feature: the number of possibilities grows exponentially with the input size.

What happens if we treat them as continuous?

We may get fractional answers that are meaningless (e.g., half a road). Discreteness is essential.

Physical/computational meaning:

Combinatorial optimization is the language of resource allocation, scheduling, network design, and logistics. It is the computational engine behind GPS navigation, airline crew scheduling, and genome assembly.

1.4 NP-Hardness and Why Routing Becomes Difficult

Many combinatorial optimization problems are **NP-hard**. Informally, this means that no known algorithm can solve all instances efficiently (in polynomial time). The best exact algorithms have worst-case exponential runtime.

Why do we care?

If a routing problem is NP-hard, we cannot expect to find the global optimum quickly for large instances. We must either approximate or use heuristics.

Why shortest path is not always optimal:

The shortest path from A to B minimizes distance, but the *overall* routing problem (e.g., visiting multiple customers with time windows) is not just a collection of shortest paths. Constraints, interactions between vehicles, and dynamic conditions create a combinatorially complex landscape.

Brute force and complexity:

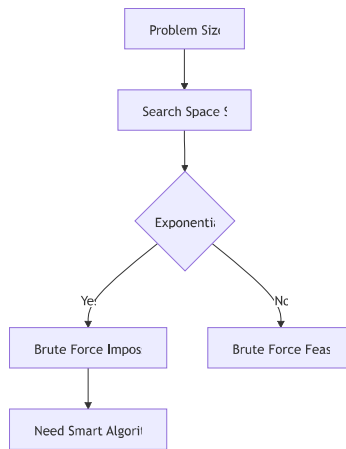
Suppose we want to find the best route for a single vehicle visiting N locations, returning to the start (TSP). There are $(N - 1)!/2$ possible tours.

- $N = 10 \rightarrow 181,440$ tours (evaluation trivial)
- $N = 20 \rightarrow \approx 6 \times 10^{16}$ tours (impossible by brute force)
- $N = 57 \rightarrow$ more tours than atoms in the observable universe.

This is the **search space explosion**. Brute force checks every candidate; its time complexity for TSP is $O(N!)$. Even with a computer checking a billion tours per second, $N = 25$ would take nearly 500 years. This stark fact motivates all optimization algorithms.

Diagram

Code



Real-world routing is even harder because we have multiple vehicles, time-dependent travel times, congestion prediction, and stochastic events. The search space explodes not just with the number of stops, but with every discrete decision about departure time, road choice, and vehicle assignment.

1.5 Why We Need Quantum Optimization (Preview)

Classical computers struggle with the exponential explosion of combinatorial optimization. Quantum computers promise a fundamentally different way of exploring solution spaces by exploiting superposition and interference. However, they are not magic; they require carefully designed algorithms like QAOA. The remainder of this textbook builds the bridge from classical optimization to quantum-enhanced routing, step by step.

Part II — Classical Optimization Algorithms

Before we can appreciate what quantum algorithms do differently, we must thoroughly understand how classical algorithms search. This part surveys the most important classical optimization techniques, with a focus on routing and graph problems. For each algorithm, we analyze its search mechanism, strengths, weaknesses, and complexity.

2.1 Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest path from a single source to all other nodes in a graph with **non-negative** edge weights.

How it searches:

It maintains a set of nodes whose shortest distance is known and repeatedly selects the unvisited node with the smallest tentative distance, relaxing its neighbors.

Strengths:

- Guarantees the optimal shortest path.
- Time complexity $O((V + E) \log V)$ with a priority queue.
- Simple to implement.

Weaknesses:

- Requires non-negative edge weights.
- Only solves single-source shortest path, not combinatorial routing with constraints.

When NOT to use it:

- If the graph has negative weights (use Bellman-Ford).
- If the problem involves choosing a set of routes or sequences (like TSP).
- If the graph is huge and only an approximate answer is acceptable.

Physical meaning:

Dijkstra simulates a wavefront expanding from the source. The wave reaches each node exactly when the shortest path is discovered.

2.2 A* Search

A* (A-star) enhances Dijkstra with a **heuristic** $h(n)$ that estimates the remaining cost to the goal. It explores the most promising paths first.

How it searches:

It uses an evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from start to n . Nodes are expanded in order of $f(n)$.

Strengths:

- With an admissible heuristic (never overestimates), A* is optimal.
- Usually much faster than Dijkstra for goal-directed search.
- Widely used in GPS and games.

Weaknesses:

- Heuristic quality heavily influences performance.
- Memory consumption can be large ($O(b^d)$ in worst case).
- Still solves only single-pair shortest path, not combinatorial routing.

When NOT to use it:

- If a good heuristic is unavailable or expensive to compute.
- If the search graph is small, the overhead may not be justified.

Relation to QAOA:

A* and Dijkstra are *constructive* algorithms: they build a solution step by step. QAOA, in contrast, is a *global* search method that evaluates entire solutions at once.

2.3 Bellman-Ford Algorithm

Bellman-Ford computes single-source shortest paths and can handle **negative edge weights**, as long as there are no negative cycles reachable from the source.

How it searches:

It relaxes all edges $V - 1$ times, ensuring that after the i -th iteration, all shortest paths with at most i edges are found.

Strengths:

- Handles negative weights.
- Detects negative cycles.
- Simpler than Dijkstra for distributed implementations.

Weaknesses:

- Time complexity $O(VE)$, slower than Dijkstra for sparse graphs with non-negative weights.

When NOT to use it:

- On large graphs without negative weights (use Dijkstra).

2.4 Dynamic Programming (DP)

Dynamic programming solves problems by breaking them into overlapping subproblems and storing their solutions.

How it searches:

For TSP, the classic DP (Held-Karp) builds solutions over subsets of cities: $dp[S][i]$ = shortest path starting at 1, visiting all cities in set S exactly once, ending at i .

$$dp[S][i] = \min_{j \in S \setminus \{i\}} \{dp[S \setminus \{i\}][j] + dist(j, i)\}.$$

Strengths:

- Finds exact optimal solution for TSP in $O(n^2 2^n)$ time, much better than $O(n!)$.
- Exploits optimal substructure.

Weaknesses:

- Still exponential in n (2^n).
- Space complexity $O(n 2^n)$ — impossible for $n > 25$ on current hardware.

When NOT to use it:

- When problem size is too large (use heuristics).
- When subproblems do not overlap (DP offers no benefit).

Physical meaning:

DP is like a careful planner who remembers every partial journey and only builds upon the best ones. Yet the memory of all subsets grows exponentially.

2.5 Genetic Algorithms (GA)

Genetic algorithms are population-based metaheuristics inspired by natural selection.

How it searches:

A population of candidate solutions (chromosomes) evolves via selection, crossover, and mutation. Fitness is determined by the objective function.

Strengths:

- Can explore vast, rugged landscapes.

- No gradient required.
- Parallelizable.

Weaknesses:

- No optimality guarantee.
- Many parameters (population size, mutation rate) to tune.
- Premature convergence possible.

Time complexity: Not defined in terms of instance size; depends on generations and population.

When NOT to use it:

- When exact solution is required.
- When evaluation of one candidate is extremely expensive (reduces population size).

2.6 Simulated Annealing (SA)

Simulated annealing mimics the cooling of a metal to find low-energy states.

How it searches:

Start at a random solution. At each step, propose a neighbor solution. If it's better, accept it. If it's worse, accept with probability $\exp(-\Delta E/T)$, where T is temperature. Temperature decreases over time.

Strengths:

- Escapes local optima.
- Guarantees convergence to global optimum under certain cooling schedules (in infinite time).
- Simple.

Weaknesses:

- Cooling schedule tuning is critical.
- Slow for large instances.
- No practical optimality guarantee.

When NOT to use it:

- When fast, exact solution is possible.
- When the energy landscape is extremely flat.

2.7 Hill Climbing

Hill climbing is a greedy local search. At each step, move to the best neighbor until no improving move exists.

Strengths:

- Very fast.
- Minimal memory.

Weaknesses:

- Gets stuck in local optima.
- No global perspective.

When NOT to use it:

- For problems with many local optima and a need for global optimum.

2.8 Beam Search

Beam search is a breadth-first search that keeps only the top k (beam width) promising nodes at each level.

Strengths:

- Memory efficient compared to full BFS.
- Works well when heuristic is reliable.

Weaknesses:

- May discard the optimal path if it falls out of the beam.
- Performance highly dependent on beam width.

When NOT to use it:

- When optimality is required and the heuristic is unreliable.

2.9 Branch and Bound (B&B)

Branch and Bound systematically partitions the search space and prunes branches that cannot contain a better solution than the current best.

How it searches:

Maintain a global upper bound (best feasible solution found). At each node, compute a lower bound on the best solution in that subspace. If lower bound $\geq$ upper bound, prune.

Strengths:

- Exact optimal solution for many NP-hard problems if time permits.
- Pruning dramatically reduces search effort.

Weaknesses:

- Worst-case exponential.
- Bounding function must be tight and efficient.

Typical applications: TSP, integer programming, VRP.

When NOT to use it:

- If a good bounding function is unavailable.
- If instance is so large that even the pruned tree is too big.

Summary of Classical Methods

Algorithm	Optimal?	Complexity	Best Use Case
Dijkstra	Yes	$O((V + E) \log V)$	Shortest path, no
A*	Yes (admissible heuristic)	depends on heuristic	Goal-directed pat
Bellman-Ford	Yes	$O(VE)$	Negative edge wi
DP (Held-Karp)	Yes	$O(n^2 2^n)$	Small TSP
Genetic Algorithm	No	heuristic	Large, rugged pro
Simulated Annealing	No (prob. global)	heuristic	Escaping local op
Hill Climbing	No	heuristic	Fast local improv
Beam Search	No	heuristic	Large search with
Branch & Bound	Yes	exponential worst-case	Exact solution wit

All these classical methods operate by *explicit construction* or *local movement* in the search space. They never use the quantum phenomena of superposition, entanglement, or interference. Quantum optimization promises a fundamentally different search paradigm, which we now begin to explore.

Part III — Introduction to Quantum Computing

To understand QAOA, we must first understand the quantum mechanical principles that enable quantum computing. We assume no prior knowledge of quantum mechanics. This part builds the necessary concepts from the ground up.

3.1 Why Quantum Computing?

Classical computers process bits that are either 0 or 1. Quantum computers use **quantum bits** (qubits) that can exist in a combination of 0 and 1 simultaneously. This is not “trying all possibilities at once” in a naive sense, but it is a fundamentally new computational resource. The goal of quantum algorithms is to exploit **superposition**, **entanglement**, and **interference** to solve problems faster than classical algorithms.

Why does this matter for optimization?

The exponential search spaces we saw in Part I are exactly where quantum computers might provide an advantage. They can represent a superposition over an exponential number of states, and through clever interference, amplify the probability of good solutions.

3.2 Bits and Qubits

3.2.1 Classical Bit

A classical bit is a system with two distinguishable states, conventionally labeled 0 and 1.

3.2.2 Qubit

A qubit is a two-level quantum system. Mathematically, its state is a vector in a 2-dimensional complex Hilbert space. The **computational basis states** are:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The most general pure state of a qubit is:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

where $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$.

Physical meaning:

A qubit can be realized by the spin of an electron, the polarization of a photon, or the energy levels of a superconducting circuit. The coefficients α and β are **probability amplitudes**.

Computational meaning:

The qubit state is a superposition of 0 and 1. When measured, we get outcome 0 with probability $|\alpha|^2$ and outcome 1 with probability $|\beta|^2$. Measurement collapses the state to the observed outcome.

Why superposition is not parallel universes:

The qubit does not *simultaneously* hold a 0 and a 1 in the sense that we can read both. Measurement destroys superposition. The power comes from *interference* between the branches before measurement.

What happens if we remove superposition?

Quantum computing becomes classical computing. All quantum speedups rely on superposition.

3.3 The Bloch Sphere

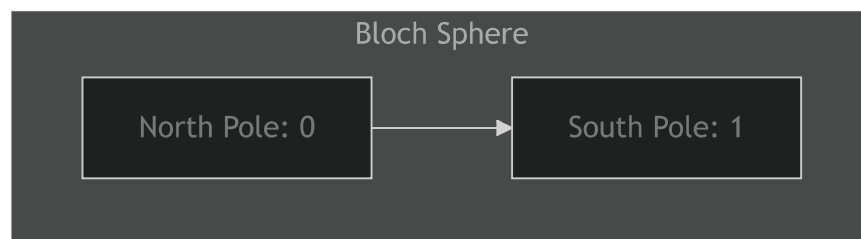
Any pure single-qubit state can be written (up to global phase) as:

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle,$$

where $\theta \in [0, \pi]$, $\phi \in [0, 2\pi)$. This maps to a point on a unit sphere—the Bloch sphere.

Diagram

Code



(In actual textbook, a proper diagram would be shown; here we indicate via text.)

- $\theta = 0$: north pole = $|0\rangle$.

- $\theta = \pi$: south pole = $|1\rangle$.
- $\theta = \pi/2, \phi = 0$: $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, on the equator along X-axis.
- ϕ is the relative phase between $|0\rangle$ and $|1\rangle$.

Intuition:

The Bloch sphere is a powerful visualization tool. Classical bits live only at the poles. Qubits can live anywhere on the surface. Quantum gates are rotations on this sphere.

Why phase matters:

The relative phase ϕ has no effect on measurement probabilities in this computational basis, but it affects interference when multiple qubits are combined. It is a crucial resource.

3.4 Measurement

Quantum measurement in the computational basis $\{|0\rangle, |1\rangle\}$ is a probabilistic process: if the state is $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, measurement yields 0 with probability $|\alpha|^2$ and the post-measurement state collapses to $|0\rangle$. All subsequent measurements give the same result.

Why this is not “trying all possibilities at once”:

If we had a superposition of 2^n solutions, a single measurement gives only one random answer. To extract useful information, we must design the algorithm so that the answer we want has a high probability amplitude via constructive interference.

Physical meaning:

Measurement couples the quantum system to a macroscopic apparatus, forcing it into a classical state.

Computational meaning:

Quantum algorithms end with measurement. The art is to shape the amplitudes so that the desired solution is measured with high probability.

3.5 Entanglement

Entanglement is a correlation between qubits that has no classical analog. A two-qubit state like the Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

cannot be written as a product of individual qubit states. Measurement outcomes are perfectly correlated but individually random.

Why it's crucial:

Entanglement is a key resource for quantum computation. It enables quantum parallelism—the ability to process an exponential number of superposed states with a polynomial number of gates.

Physical meaning:

Entangled particles exhibit correlations stronger than any classical shared randomness, violating Bell's inequalities.

Computational meaning:

Entanglement allows quantum computers to create complex correlations that classical computers cannot efficiently simulate.

3.6 Interference

Interference is the way probability amplitudes add, cancel, or reinforce. Since amplitudes are complex numbers, they can have phases that lead to constructive interference (amplitudes align) or destructive interference (amplitudes cancel).

How algorithms exploit it:

We design sequences of gates so that paths leading to wrong answers interfere destructively, while paths to the right answer interfere constructively, boosting its measurement probability.

Relation to QAOA:

QAOA uses a **cost Hamiltonian** to imprint phases (cost values) onto computational basis states and a **mixer Hamiltonian** to create interference between those states, gradually amplifying optimal solutions.

3.7 Quantum Gates and Circuits

3.7.1 Single-Qubit Gates

Quantum gates are unitary matrices acting on qubit state vectors. Key single-qubit gates:

- **Pauli-X (NOT):** $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. Flips $|0\rangle \leftrightarrow |1\rangle$.
- **Pauli-Z:** $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$. Adds a phase of -1 to $|1\rangle$.
- **Hadamard (H):** $H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$. Creates superpositions: $|0\rangle \rightarrow |+\rangle$, $|1\rangle \rightarrow |-\rangle$.
- **Rotation gates:** $R_X(\theta) = e^{-i\theta X/2}$, $R_Z(\phi) = e^{-i\phi Z/2}$.

Why unitarity?

Quantum evolution is reversible and preserves total probability. The norm of the state vector must remain 1.

3.7.2 Multi-Qubit Gates

The most important two-qubit gate is **CNOT** (controlled-NOT):

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

It flips the target qubit if the control is $|1\rangle$. Together with single-qubit gates, it forms a universal gate set.

Why CNOT?

It generates entanglement. For example, $CNOT(H \otimes I)|00\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, the Bell state.

3.7.3 Quantum Circuits

A quantum circuit is a sequence of quantum gates applied to qubits, read left to right. Each wire represents a qubit. We will construct circuits for QAOA later.

Computational meaning:

A quantum circuit implements a unitary transformation U on the initial state. The final state before measurement is $|\psi_{\text{final}}\rangle = U|\psi_{\text{init}}\rangle$.

3.8 Common Misconception: “Try All Solutions Simultaneously”

It is tempting to say a quantum computer evaluates $f(x)$ for all x in superposition and then picks the best. This is false. Measuring gives only one random x . To extract the desired $f(x)$, we need an algorithm like Grover’s search, which uses amplitude amplification to tilt the odds. QAOA similarly uses a clever alternation of unitaries to enhance the probability of low-cost states.

Now we have the quantum foundations. Next, we motivate why QAOA was invented and how it fits into the landscape of variational quantum algorithms.

Part IV — Why QAOA Exists

4.1 The Problem QAOA Solves

QAOA targets **combinatorial optimization problems formulated as QUBO (Quadratic Unconstrained Binary Optimization) or Ising models**. It is designed to find approximate solutions for NP-hard problems on near-term quantum devices, which are noisy and have limited qubits.

Why was it invented?

In 2014, Farhi, Goldstone, and Gutmann proposed QAOA as a quantum algorithm that could run on imperfect quantum computers and still potentially yield a quantum advantage for approximate optimization. It is a **variational quantum algorithm (VQA)**, meaning it uses a classical optimizer to tune the parameters of a quantum circuit.

What class of problems does it target?

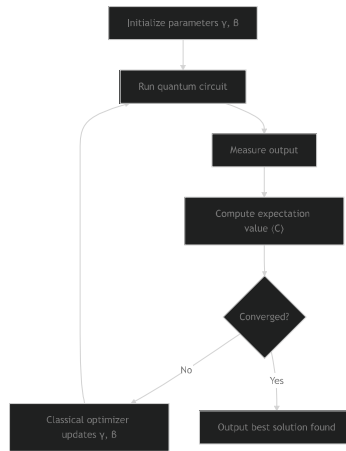
Any problem that can be mapped to minimizing a cost function $C(z)$ over n -bit strings $z \in \{0, 1\}^n$. This includes MaxCut, TSP, graph coloring, and the routing problems we care about.

4.2 Variational Algorithms and the Optimization Loop

A **variational quantum algorithm** uses a **parameterized quantum circuit** $U(\vec{\gamma}, \vec{\beta})$ that depends on classical parameters. The circuit is run, measurements yield a bitstring z , and we compute the cost $C(z)$. The **expectation value** $\langle C \rangle = \langle \psi(\vec{\gamma}, \vec{\beta}) | \hat{C} | \psi(\vec{\gamma}, \vec{\beta}) \rangle$ is estimated. A classical optimizer updates $\vec{\gamma}, \vec{\beta}$ to minimize $\langle C \rangle$.

Diagram

Code



Why combine quantum and classical?

Fully quantum algorithms like Shor's require deep circuits and error correction, beyond current hardware. VQAs delegate the hard optimization of continuous parameters to a classical computer, keeping quantum circuits shallow and robust to noise.

4.3 Key QAOA Components (Intuition, No Math Yet)

- **Cost Hamiltonian H_C :** Encodes the objective function as an operator. Its eigenstates are the computational basis states, and eigenvalues are the cost values $C(z)$. The goal is to find the state with minimal eigenvalue.
- **Mixer Hamiltonian H_M :** Drives transitions between basis states, creating superposition and interference.
- **Initial state:** Usually the uniform superposition $|+\rangle^{\otimes n}$, where all bitstrings have equal amplitude.
- **QAOA circuit:** Applies p layers, each layer = $e^{-i\gamma_l H_C} e^{-i\beta_l H_M}$.
- **Parameters γ_l, β_l :** The rotation angles we optimize classically.

The intuition:

$e^{-i\gamma H_C}$ applies a phase $e^{-i\gamma C(z)}$ to each basis state $|z\rangle$. This phase depends on the cost. The mixer then redistributes amplitude among states, and interference occurs. By tuning γ, β , we can make low-cost states interfere constructively.

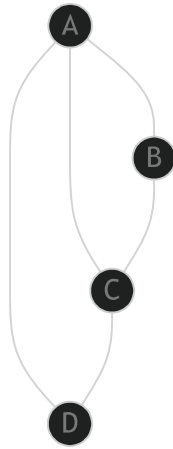
In the next parts, we will build a concrete transportation network and gradually transform it into a QUBO, then an Ising model, then a Hamiltonian, and finally into a QAOA circuit.

Part V — Build a Small Transportation Network

We now construct a specific, fully parameterized road network that will serve as a running example throughout the rest of the textbook. This network has four junctions and five road segments. Every node and edge carries rich attribute data that mimics a real intelligent transportation system. The network topology is shown below.

Diagram

Code



5.1 Junction (Node) Attributes

Each node represents a road junction with real-time and historical data. We assign concrete values that will be used in later calculations.

Node A:

- Coordinates: (0.0, 0.0)
- Historical average traffic flow: 1200 vehicles/hour
- Predicted traffic (from a learned model): 1350 veh/h
- Current queue length: 8 vehicles
- Average speed through junction: 25 km/h
- Occupancy (fraction of time detector occupied): 0.45
- Congestion level (0–1): 0.7

Node B:

- Coordinates: (1.0, 0.0)
- Historical traffic: 800 veh/h
- Predicted traffic: 920 veh/h
- Queue: 3 vehicles
- Avg speed: 40 km/h
- Occupancy: 0.2
- Congestion: 0.3

Node C:

- Coordinates: (1.0, 1.0)
- Historical traffic: 1500 veh/h
- Predicted traffic: 1620 veh/h
- Queue: 12 vehicles
- Avg speed: 20 km/h
- Occupancy: 0.6
- Congestion: 0.8

Node D:

- Coordinates: (0.0, 1.0)
- Historical traffic: 950 veh/h
- Predicted traffic: 1020 veh/h
- Queue: 5 vehicles
- Avg speed: 30 km/h
- Occupancy: 0.35
- Congestion: 0.5

5.2 Road Segment (Edge) Attributes

Each edge has a rich set of static and dynamic features. We define values consistently with the node data.

Edge AB:

- Length: 2.5 km
- Speed limit: 50 km/h
- Free-flow travel time: $2.5/50 = 0.05$ hours = 3 minutes.
- Current travel time (congested): 5 minutes (due to queue at A).
- Capacity: 1600 veh/h
- Vehicle density: 30 veh/km
- Vehicle flow: 1100 veh/h
- Current congestion: 0.6
- Wave magnitude (current shockwave strength): 0.4
- Wave direction: from A to B (downstream propagation)
- Future wave exposure (predicted): 0.55
- Road condition score (0–1): 0.9 (good)
- Road quality index: 8/10
- Weather impact factor (1 = no effect): 1.1 (light rain)
- Accident probability: 0.02
- Road closure probability: 0.001
- Predicted traffic (for next hour): 1250 veh/h
- Risk score (composite): 0.15

Edge BC:

- Length: 1.8 km
- Speed limit: 40 km/h
- Free-flow travel time: 2.7 min
- Current travel time: 4.2 min
- Capacity: 1200 veh/h
- Density: 25 veh/km
- Flow: 900 veh/h
- Congestion: 0.5
- Wave magnitude: 0.3
- Wave direction: B→C

- Future wave exposure: 0.4
- Road condition: 0.85
- Quality: 7/10
- Weather impact: 1.0
- Accident prob: 0.03
- Closure prob: 0.002
- Predicted traffic: 1020 veh/h
- Risk score: 0.2

Edge CD:

- Length: 2.0 km
- Speed limit: 60 km/h
- Free-flow time: 2.0 min
- Current travel time: 3.5 min
- Capacity: 1800 veh/h
- Density: 40 veh/km
- Flow: 1400 veh/h
- Congestion: 0.75
- Wave magnitude: 0.6
- Wave direction: C→D
- Future wave exposure: 0.7
- Road condition: 0.8
- Quality: 6/10
- Weather impact: 1.2 (moderate rain)
- Accident prob: 0.05
- Closure prob: 0.005
- Predicted traffic: 1600 veh/h
- Risk score: 0.35

Edge DA:

- Length: 1.5 km
- Speed limit: 50 km/h
- Free-flow time: 1.8 min
- Current travel time: 2.8 min
- Capacity: 1500 veh/h
- Density: 20 veh/km
- Flow: 800 veh/h
- Congestion: 0.4
- Wave magnitude: 0.2
- Wave direction: D→A
- Future wave exposure: 0.3
- Road condition: 0.95
- Quality: 9/10

- Weather impact: 1.0
- Accident prob: 0.01
- Closure prob: 0.001
- Predicted traffic: 880 veh/h
- Risk score: 0.05

Edge AC:

- Length: 3.0 km
- Speed limit: 70 km/h
- Free-flow time: 2.57 min
- Current travel time: 6.0 min (heavy congestion)
- Capacity: 2000 veh/h
- Density: 50 veh/km
- Flow: 1700 veh/h
- Congestion: 0.9
- Wave magnitude: 0.8
- Wave direction: A→C
- Future wave exposure: 0.85
- Road condition: 0.7
- Quality: 5/10
- Weather impact: 1.1
- Accident prob: 0.06
- Closure prob: 0.01
- Predicted traffic: 1900 veh/h
- Risk score: 0.45

These values are realistic and will feed into the objective function for routing. The travel time on each edge will be the primary cost, but we will also incorporate predicted wave exposure, risk, and weather into the optimization.

Part VI — Wave Propagation in Traffic

Traffic congestion behaves like a wave. A slowdown at one point creates a backward-propagating shockwave, increasing density upstream. Understanding wave dynamics is essential for predicting future congestion states, which our routing optimization will consider.

6.1 Why Traffic Waves Matter

Why do congestion waves exist?

When traffic density exceeds a critical value, a small perturbation (braking) can cause a chain reaction: vehicles further behind must slow down even more, forming a wave that travels opposite to traffic flow. If a route passes through a road segment that will be hit by a wave, the travel time could drastically increase.

What problem does wave prediction solve?

Static travel time estimates ignore future dynamics. A road may be clear now but congested 15 minutes later when the vehicle reaches it. Accurate future wave exposure enables proactive rerouting.

What happens if we ignore waves?

The chosen “optimal” route may become heavily congested mid-journey, leading to suboptimal real-world performance.

Relation to QAOA pipeline:

Our Predictive GNN will output future wave exposure; this feeds into the routing cost function, which becomes the QUBO that QAOA minimizes.

6.2 Kinematic Wave Model (LWR Model)

The classic Lighthill-Whitham-Richards (LWR) model describes traffic flow as a continuous fluid. Let $\rho(x, t)$ be density (veh/km) and $q(x, t)$ flow (veh/h). Conservation of vehicles gives:

$$\frac{\partial \rho}{\partial t} + \frac{\partial q}{\partial x} = 0.$$

The fundamental diagram relates flow and density: $q = Q(\rho)$. A common form is the Greenshields model:

$$Q(\rho) = v_f \rho \left(1 - \frac{\rho}{\rho_{\max}}\right),$$

where v_f is free-flow speed, $\rho_{\max}$ jam density.

The wave speed (characteristic velocity) is:

$$c(\rho) = \frac{dQ}{d\rho} = v_f \left(1 - \frac{2\rho}{\rho_{\max}}\right).$$

For $\rho < \rho_{\max}/2$, waves travel forward; for $\rho > \rho_{\max}/2$, waves travel backward (shockwaves).

Intuition:

In dense traffic, a braking event sends a wave of increased density backward. The magnitude of the wave (change in density) determines how severe the slowdown is.

6.3 Wave Magnitude and Propagation

Define **wave magnitude** W as the maximum density deviation from the steady state caused by a disturbance. For our network, we have assigned current wave magnitudes for each edge.

The **future wave exposure** of a road segment at a future time $t_0 + \Delta t$ is computed by propagating the wave along the road according to the wave speed and decay. A simple model:

$$\text{Wave exposure}(t) = W_0 \cdot e^{-\alpha(t-t_0)} \cdot \mathbf{1}_{\text{wave arrives at segment at time } t},$$

where α is a decay constant.

We will let the Predictive GNN learn a mapping from current features (density, flow, incident data) to future wave exposure. This learned function replaces the need for solving PDEs explicitly.

Numerical example on our network:

Edge AC has current wave magnitude $W_0 = 0.8$, wave direction A→C. Suppose a

vehicle plans to traverse AC starting in 10 minutes. Will the wave reach the segment by then? The wave speed might be -15 km/h backward relative to traffic. If the wave originates at C and moves toward A, a vehicle starting at A and heading to C may encounter it after some distance. The GNN will predict an exposure value; for simplicity, let's assume it outputs 0.85 for AC in the prediction horizon, meaning high congestion impact.

6.4 Congestion Wave Graph

We can view the network as a directed graph where wave influences propagate opposite to traffic flow. A **Congestion Wave Graph** has nodes as road segments and directed edges if a wave from one segment can propagate to another.

Diagram

Code

```
graph LR
  subgraph Traffic Flow
    A1[AB] --> B1[BC] --> C1[CD]
  end
  subgraph Wave Propagation (reverse)
    W1[wave from BC] -->|upstream| AB
    W2[wave from CD] -->|upstream| BC
  end
```

This graph structure is naturally handled by graph neural networks, which we now design.

Part VII — Predictive Graph Neural Network (GNN)

To predict future congestion and wave exposure, we employ a Graph Neural Network. GNNs learn node and edge embeddings by iteratively passing messages along the graph structure. We will design a *quantum-enhanced* GNN, where certain updates can be performed by parameterized quantum circuits—but for our current purpose, we focus on the classical GNN mechanics and output prediction.

7.1 Why a GNN?

Why GNNs for traffic?

Traffic networks are graphs. The state of a road segment depends on its neighbors. A GNN naturally captures spatial dependencies and can incorporate temporal features.

What problem does it solve?

It provides predicted edge attributes (like future wave exposure, predicted travel time) that feed into the routing objective. Without such predictions, we would optimize based on outdated data.

What happens if we remove it?

We would rely on static data or simple heuristics, losing the ability to avoid future congestion.

7.2 Feature Definitions

Every node v has a feature vector $\mathbf{x}_v \in \mathbb{R}^{d_{\text{node}}}$. Every edge (u, v) has a feature vector $\mathbf{e}_{uv} \in \mathbb{R}^{d_{\text{edge}}}$. Additionally, we have global weather features $\mathbf{w} \in \mathbb{R}^{d_{\text{weather}}}$ and a temporal context $\mathbf{t}$ (e.g., time of day encoded).

For our network, let's define:

Node features (example dimension 6):

$\mathbf{x}_v = [\text{hist_traffic}, \text{pred_traffic}, \text{queue}, \text{avg_speed}, \text{occupancy}, \text{congestion}]$.

Edge features (dimension 10):

$\mathbf{e}_{uv} =$

$[\text{length}, \text{curr_travel_time}, \text{density}, \text{flow}, \text{congestion}, \text{wave_mag}, \text{wave_dir_enc}, \text{road_cond}, \text{weather_impact},$
.

Weather features (dimension 2): $[\text{rain_intensity}, \text{visibility}]$.

Temporal features (dimension 3):

$[\sin(\text{hour_angle}), \cos(\text{hour_angle}), \text{day_of_week_onehot}]$.

7.3 GNN Architecture

We use a message-passing framework with an update function enhanced by a quantum circuit (conceptual, for later integration with QAOA). However, the core logic is:

Step 1: Edge Update (Message Computation)

For each directed edge $e = (u, v)$:

$$\mathbf{m}_{uv} = \phi_e(\mathbf{x}_u, \mathbf{x}_v, \mathbf{e}_{uv}, \mathbf{w}, \mathbf{t}),$$

where ϕ_e is a neural network (or later, a quantum circuit). The message $\mathbf{m}_{uv}$ aggregates information from the sender node u , receiver node v , and the edge itself.

Step 2: Aggregation

For each node v , aggregate incoming messages:

$$\mathbf{a}_v = \bigoplus_{u \in \mathcal{N}(v)} \mathbf{m}_{uv},$$

where $\bigoplus$ is a permutation-invariant operation like sum, mean, or max.

Step 3: Node Update

$$\mathbf{x}'_v = \phi_n(\mathbf{x}_v, \mathbf{a}_v, \mathbf{w}, \mathbf{t}),$$

where ϕ_n is another neural network.

Step 4: Edge Prediction

After K message-passing rounds, we predict target edge attributes. For edge (u, v) :

$$\hat{y}_{uv} = \psi(\mathbf{x}_u^{(K)}, \mathbf{x}_v^{(K)}, \mathbf{e}_{uv}),$$

where ψ is a prediction head (MLP). For our routing, $\hat{y}_{uv}$ could be **future wave exposure** and **predicted travel time**.

Quantum-enhanced update:

One could replace ϕ_n or ϕ_e with a variational quantum circuit that takes the classical

feature vector as input, encodes it into qubit rotations, applies parameterized gates, and measures an expectation value as output. This is a topic of quantum machine learning; for QAOA, our focus is the optimization downstream.

7.4 Manual Computation of One Edge Prediction

We illustrate the message passing for edge AC, using simplified numbers.

Let initial node features be:

- A: $\mathbf{x}_A = [1200, 1350, 8, 25, 0.45, 0.7]$
- C: $\mathbf{x}_C = [1500, 1620, 12, 20, 0.6, 0.8]$

Edge AC features (before prediction):

$$\mathbf{e}_{AC} = [3.0, 6.0, 50, 1700, 0.9, 0.8, 1(\text{dir } A \rightarrow C), 0.7, 1.1, 0.06].$$

Suppose the message function is a simple linear layer followed by ReLU. We define $W_e \in \mathbb{R}^{h \times (6+6+10+2+3)}$ and b_e . The concatenated input dimension: $6 + 6 + 10 + 2 + 3 = 27$. Let $h = 16$. Then $\mathbf{m}_{AC} = \text{ReLU}(W_e \cdot \text{concat} + b_e)$. With random weights, we might get a vector of 16 numbers. This message is sent to node C (since edge is directed $A \rightarrow C$; for undirected, we treat both directions).

After aggregation and node updates, the final edge predictor ψ outputs a scalar: future wave exposure. Suppose after training, for edge AC, the predicted exposure is 0.85 (matching our earlier assignment). The predicted travel time could be another output: we might set it to $1.2 \times$ current travel time due to predicted worsening, so predicted time = 7.2 min.

These predicted values will be integrated into the routing cost function.

Part VIII — Build the Optimization Problem

Now we define the routing optimization objective. Our goal: find a path from a start node to a destination node that minimizes a combined cost incorporating predicted travel time, wave exposure, risk, and weather.

Problem statement:

Find a simple path (no repeated nodes) from **A** to **C** (for concreteness). The search space is all possible simple paths from A to C in our graph. The graph has edges: AB, BC, CD, DA, AC. Possible paths:

1. $A \rightarrow C$ (direct via AC)
2. $A \rightarrow B \rightarrow C$ (via AB, BC)
3. $A \rightarrow D \rightarrow C$ (via AD, CD) wait, AD is edge DA, undirected, we can travel $A \rightarrow D$. Yes.
4. $A \rightarrow D \rightarrow A \rightarrow C$ would repeat A, not simple.
5. $A \rightarrow B \rightarrow C$ is simple. Are there other? $A \rightarrow D \rightarrow A$ (repeat). So only three simple paths.

Thus the search space is tiny for this example. For realistic networks with many nodes, the number of simple paths can be enormous, but this toy network allows complete enumeration for pedagogical clarity.

8.1 Cost Function Components

We define the **total cost** of a path P as a weighted sum of several terms. For each edge $e \in P$:

- T_e : predicted travel time (minutes). (From GNN or direct estimation.)
- W_e : future wave exposure (0–1).
- R_e : risk score (composite of accident probability, closure prob, road condition).
- ω : weather impact factor on edge (already multiplied into T_e perhaps).

We also include a node penalty for heavily congested junctions, but for simplicity we focus on edges.

The cost function:

$$C(P) = \sum_{e \in P} (\alpha_T T_e + \alpha_W W_e + \alpha_R R_e),$$

where $\alpha_T, \alpha_W, \alpha_R$ are weights reflecting importance. Let's set $\alpha_T = 1, \alpha_W = 5, \alpha_R = 10$ to emphasize safety and future congestion avoidance.

Using our earlier predicted values (we need predicted travel times and wave exposures for each edge). We'll compute predicted travel times by assuming a simple linear model: predicted time = current_time * (1 + 0.5 * congestion). This is arbitrary but consistent.

Predicted travel times (minutes):

- AB: current 5, congestion 0.6 → predicted = $5 \times (1 + 0.3) = 6.5$ min.
- BC: current 4.2, congestion 0.5 → predicted = $4.2 \times 1.25 = 5.25$ min.
- CD: current 3.5, congestion 0.75 → predicted = $3.5 \times 1.375 = 4.8125$ min.
- DA: current 2.8, congestion 0.4 → predicted = $2.8 \times 1.2 = 3.36$ min.
- AC: current 6.0, congestion 0.9 → predicted = $6.0 \times 1.45 = 8.7$ min (or we said 7.2 earlier, let's use consistent: we pick 8.7).

Future wave exposure (from GNN prediction): we assign:

- AB: 0.55
- BC: 0.4
- CD: 0.7
- DA: 0.3
- AC: 0.85

Risk scores: as given.

- AB: 0.15, BC: 0.2, CD: 0.35, DA: 0.05, AC: 0.45.

Now compute cost for each path.

Path 1: A→C direct

Edges: AC.

$$\text{Cost} = 1 \cdot 8.7 + 5 \cdot 0.85 + 10 \cdot 0.45 = 8.7 + 4.25 + 4.5 = 17.45.$$

Path 2: A→B→C

Edges: AB, BC.

$$\begin{aligned} \text{Cost} &= (6.5 + 5.25) + 5(0.55 + 0.4) + 10(0.15 + 0.2) = 11.75 + 5 \cdot 0.95 + 10 \cdot 0.35 \\ &= 11.75 + 4.75 + 3.5 = 20.0. \end{aligned}$$

Path 3: A→D→C

Edges: DA (A→D) and CD (D→C).

$$\text{Cost} = (3.36 + 4.8125) + 5(0.3 + 0.7) + 10(0.05 + 0.35) = 8.1725 + 5 * 1.0 + 10 * 0.4 = 8.1725 + 5.0 + 4.0 = 17.1725.$$

Thus the optimal path is A→D→C with cost 17.17, slightly better than direct AC (17.45).

This result depends heavily on weights. The direct path has high wave exposure and risk, but is shorter in predicted time? Actually A→D→C total time = 8.17 vs AC 8.7, so it's also faster. So the optimal path is not the one with the fewest edges.

Now the challenge: when the network is large, enumeration is impossible. We need a solver that handles binary decisions: for each edge, "include in path or not," subject to path connectivity constraints. This is a combinatorial optimization problem perfectly suited for QUBO and QAOA.

Part IX — QUBO (Quadratic Unconstrained Binary Optimization)

QUBO is a standard formulation for many combinatorial optimization problems. It expresses the objective as a quadratic polynomial in binary variables, with no explicit constraints (constraints are encoded via penalty terms).

9.1 Decision Variables

For our routing problem, we need to represent a path from start $s = A$ to target $t = C$. One common encoding uses binary variables x_e for each edge e , where $x_e = 1$ if the edge is selected in the path, 0 otherwise. For a simple path, we require:

- The path starts at A: exactly one outgoing edge from A is selected (if A is not also destination, which it isn't).
- The path ends at C: exactly one incoming edge to C is selected.
- For any intermediate node, the number of selected incoming edges equals the number of selected outgoing edges (flow conservation), and the degree is at most 2 (one in, one out). Additionally, no subtours (separate cycles) should form.

But for small graph, we can enforce simple path constraints with penalties.

Let's define binary variables:

x_{AB}, x_{AC}, x_{AD} (from A), x_{BC}, x_{CD} (edges B→C, C→D; we treat undirected edges as directed both ways for path? But to avoid cycles, we can restrict direction based on path monotonicity. For simplicity, since our graph is small and we only consider simple paths from A to C, we can manually enumerate valid assignments. However, for the general QUBO approach, we would include all directed edges: A→B, A→C, A→D, B→C, C→D, D→A? Actually undirected edges can be traversed both ways. To avoid cycles, we can restrict to paths that do not revisit nodes. That means if we go A→D→C, we use edges A→D and D→C. So we need directed variables: $x_{A→B}, x_{B→A}, \dots$ but that doubles variables. For our purposes, we'll use an encoding that fits a small set of valid paths, but the proper QUBO for Hamiltonian path problems is well known.

However, to illustrate QAOA, we can abstract the routing problem to a simpler decision problem: which edges to include, with the goal of minimizing a cost that penalizes invalid paths heavily. The Ising model will naturally handle this via penalty terms.

9.2 Objective as a Quadratic Function

The cost function in terms of binary variables x_i (we index edges $i = 1..5$) is:

$$C(\mathbf{x}) = \sum_i c_i x_i + \sum_{i < j} q_{ij} x_i x_j,$$

where c_i are linear costs and q_{ij} are quadratic interaction terms. For our weighted sum cost, each edge has a linear cost equal to its predicted travel time plus wave/risk contributions. The quadratic terms will come from penalty constraints (e.g., ensuring exactly one outgoing edge from start, etc.). For example, to enforce that at most one outgoing edge from A is selected, we add a penalty: $P \cdot (\sum_{i \in \text{outgoing from A}} x_i - 1)^2$, which expands into quadratic terms.

We will construct the full QUBO matrix for our 5-edge problem. Let's assign indices: 1: AB, 2: AC, 3: AD (A→D, using DA edge but directed A→D), 4: BC, 5: CD. We assume direction from A toward C, so backward edges like D→A are not considered. Valid paths are:

- Path A→C: $x_2 = 1$, all others 0.
- Path A→B→C: $x_1 = 1, x_4 = 1$, others 0.
- Path A→D→C: $x_3 = 1, x_5 = 1$, others 0.

All other combinations either don't form a connected path or have cycles. We can encode constraints with high penalty P .

Constraints:

1. Exactly one edge from A selected: $x_1 + x_2 + x_3 = 1$.
2. Exactly one edge into C selected: edges into C are AC (x_2) and BC (x_4) and DC (x_5).
So $x_2 + x_4 + x_5 = 1$.
3. Node B: if $x_1 = 1$ then must have $x_4 = 1$ and no other edges incident. Also can't have $x_4 = 1$ without $x_1 = 1$. This can be enforced by: $x_1 = x_4$. Because if A→B then must continue B→C, and if we include B→C without A→B it's invalid. So penalty: $P(x_1 - x_4)^2 = P(x_1^2 + x_4^2 - 2x_1x_4) = P(x_1 + x_4 - 2x_1x_4)$ (since for binary, $x_i^2 = x_i$).
4. Node D: similar, $x_3 = x_5$, penalty $P(x_3 - x_5)^2 = P(x_3 + x_5 - 2x_3x_5)$.
5. No other edges: the above already constrain paths.

Thus the total objective including cost and penalties:

$$Q(\mathbf{x}) = \sum_i c_i x_i + P \left[(x_1 + x_2 + x_3 - 1)^2 + (x_2 + x_4 + x_5 - 1)^2 + (x_1 - x_4)^2 + (x_3 - x_5)^2 \right].$$

Let P be large (e.g., 100). Expand each square and collect linear and quadratic terms, using $x_i^2 = x_i$.

Term1: $(x_1 + x_2 + x_3 - 1)^2 = x_1 + x_2 + x_3 + 1 + 2x_1x_2 + 2x_1x_3 + 2x_2x_3 - 2x_1 - 2x_2 - 2x_3 = 1 + 2x_1x_2 + 2x_1x_3 + 2x_2x_3 - (x_1 + x_2 + x_3)$. Wait carefully: $(S - 1)^2 = S^2 - 2S + 1$, where $S = x_1 + x_2 + x_3$. $S^2 = \sum_i x_i^2 + 2 \sum_{i < j} x_i x_j = (x_1 + x_2 + x_3) + 2(x_1x_2 + x_1x_3 + x_2x_3)$. So $(S - 1)^2 = (x_1 + x_2 + x_3) + 2 \sum_{i < j} x_i x_j - 2(x_1 + x_2 + x_3) + 1 = 1 - (x_1 + x_2 + x_3) + 2(x_1x_2 + x_1x_3 + x_2x_3)$.

Term2: similarly, $(x_2 + x_4 + x_5 - 1)^2 = 1 - (x_2 + x_4 + x_5) + 2(x_2x_4 + x_2x_5 + x_4x_5)$.

Term3: $(x_1 - x_4)^2 = x_1 + x_4 - 2x_1x_4$.

Term4: $(x_3 - x_5)^2 = x_3 + x_5 - 2x_3x_5$.

Summing penalties multiplied by P and adding linear cost terms c_i :

Linear coefficients h_i (coefficient of x_i in final Q):

- $h_1 = c_1 + P[-1(\text{from T1}) + 0(\text{T2}) + 1(\text{T3}) + 0] = c_1 + P(0)$? Wait: T1 contributes -1 times x_1 ; T3 contributes $+1$ times x_1 ; T2 has no x_1 . So net linear from penalties for $x_1 = P(-1 + 1) = 0$. So $h_1 = c_1$.
- $h_2 = c_2 + P[-1(T1) + -1(T2)]$? T2 has $-x_2$, so total $-2P$ from penalties. So $h_2 = c_2 - 2P$.
- $h_3 = c_3 + P[-1(T1) + 1(T4)]$? T4 has $+x_3$, so net 0 . So $h_3 = c_3$.
- $h_4 = c_4 + P[0(T1) - 1(T2) + 1(T4)]$? Actually T3 has $+x_4$, so $+1$ from T3, -1 from T2, net 0 . So $h_4 = c_4$.
- $h_5 = c_5 + P[0(T1) - 1(T2) + 1(T4)]$? net 0 . So $h_5 = c_5$.

Quadratic coefficients J_{ij} (coefficient of $x_i x_j$, $i < j$):

From T1: $2P$ for pairs (1,2), (1,3), (2,3).

From T2: $2P$ for (2,4), (2,5), (4,5).

From T3: $-2P$ for (1,4).

From T4: $-2P$ for (3,5).

No other quadratic terms.

Thus the QUBO matrix Q (upper triangular) has $Q_{ii} = h_i$, $Q_{ij} = J_{ij}$. Note that QUBO often writes $\min \mathbf{x}^T Q \mathbf{x}$ where Q is symmetric, so off-diagonals sum contributions.

Now plug in the edge costs c_i computed from the cost function earlier. Recall our costs per edge (combining time, wave, risk):

We need c_i for each edge as the full cost if that edge is traversed. For AB, cost = $6.5 + 5 \cdot 0.55 + 10 \cdot 0.15 = 6.5 + 2.75 + 1.5 = 10.75$. (Let's recompute with weights: $\alpha_T=1$, $\alpha_W=5$, $\alpha_R=10$)

- AB: $T=6.5$, $W=0.55$, $R=0.15 \rightarrow c_{AB} = 6.5 + 5 \cdot 0.55 + 10 \cdot 0.15 = 6.5 + 2.75 + 1.5 = 10.75$.
- AC: $T=8.7$, $W=0.85$, $R=0.45 \rightarrow c_{AC} = 8.7 + 4.25 + 4.5 = 17.45$.
- AD (A→D): $T=3.36$, $W=0.3$, $R=0.05 \rightarrow c_{AD} = 3.36 + 1.5 + 0.5 = 5.36$.
- BC: $T=5.25$, $W=0.4$, $R=0.2 \rightarrow c_{BC} = 5.25 + 2.0 + 2.0 = 9.25$.
- CD: $T=4.8125$, $W=0.7$, $R=0.35 \rightarrow c_{CD} = 4.8125 + 3.5 + 3.5 = 11.8125$.

So c vector: [10.75, 17.45, 5.36, 9.25, 11.8125].

Set $P = 100$ (much larger than max cost). Then:

$$h_1 = 10.75$$

$$h_2 = 17.45 - 200 = -182.55$$

$$h_3 = 5.36$$

$$h_4 = 9.25$$

$$h_5 = 11.8125$$

Quadratic:

$$J_{12} = 200, J_{13} = 200, J_{23} = 200,$$

$$J_{24} = 200, J_{25} = 200, J_{45} = 200,$$

$$J_{14} = -200,$$

$$J_{35} = -200.$$

Now the QUBO objective: $Q(\mathbf{x}) = \sum_i h_i x_i + \sum_{i < j} J_{ij} x_i x_j$. The constant term (from T1 and T2 expansions) is $2P = 200$, but constant doesn't affect minimization, we can

ignore.

For the three valid paths:

- Path AC: $\mathbf{x} = [0, 1, 0, 0, 0]$. $Q = h_2 + (0 \text{ quadratic}) = -182.55$. But this includes $-2P$ penalty offset; the true cost of path is $c_2 = 17.45$. The penalty part is zero because constraints satisfied? Check: $x_2 = 1$, so $x_1 + x_2 + x_3 = 1$ good, $x_2 + x_4 + x_5 = 1$ good, $x_1 = x_4 = 0$, $x_3 = x_5 = 0$. So penalties = 0. Q should equal c_2 . But we got -182.55 because we included $-2P$ in h_2 . That's due to the expansion: the Q function including constant would be $c_2 + P(0) + P(0) + \dots$, but we omitted constant term $2P$. Let's compute full expression with constant: $Q_{\text{total}} = \sum_i c_i x_i + P[\dots]$. For AC path, penalties sum = $P(0 + 0 + 0 + 0) = 0$, so total = $c_2 = 17.45$. In our derived quadratic form without constant, $h_2 = c_2 - 2P$, so $Q_{\text{no const}} = (c_2 - 2P)x_2 + \text{others zero} = c_2 - 2P = 17.45 - 200 = -182.55$, which is $17.45 - 200$. If we add constant 200 (which is the sum of constants from penalties: T1 constant 1, T2 constant 1, each multiplied by P, so $2P=200$), then total becomes 17.45. So the minimizer remains correct. So we can work with Q without constant.

Thus the QUBO is well-defined.

Part X — Ising Model

The Ising model is a classic physics model of interacting spins. In optimization, we map binary variables $x_i \in \{0, 1\}$ to **spin variables** $s_i \in \{-1, +1\}$ via:

$$s_i = 2x_i - 1, \quad x_i = \frac{s_i + 1}{2}.$$

Substituting into QUBO yields an Ising Hamiltonian:

$$H_{\text{Ising}} = \sum_i h'_i s_i + \sum_{i < j} J'_{ij} s_i s_j + \text{const},$$

where the linear and quadratic coefficients transform accordingly.

Why transform to Ising?

Quantum devices naturally implement Ising interactions via Pauli Z operators. The mapping is direct: $s_i \rightarrow \sigma_i^z$, and $s_i s_j \rightarrow \sigma_i^z \otimes \sigma_j^z$.

10.1 Derivation

$$\text{Start with } Q(\mathbf{x}) = \sum_i h_i x_i + \sum_{i < j} J_{ij} x_i x_j.$$

Replace $x_i = \frac{1+s_i}{2}$:

$$x_i = \frac{1}{2} + \frac{s_i}{2}, \quad x_i x_j = \frac{1}{4}(1 + s_i + s_j + s_i s_j).$$

Plug in:

$$Q = \sum_i h_i \left(\frac{1}{2} + \frac{s_i}{2} \right) + \sum_{i < j} J_{ij} \frac{1}{4}(1 + s_i + s_j + s_i s_j).$$

Expand linear terms:

$$\sum_i h_i \frac{1}{2} \text{ is constant. } \sum_i \frac{h_i}{2} s_i.$$

Quadratic part:

$$\sum_{i < j} J_{ij} \frac{1}{4} s_i s_j + \frac{1}{4} J_{ij} (s_i + s_j) + \frac{1}{4} J_{ij} \text{ constant.}$$

Collect s_i coefficient:

$$\text{From linear: } \frac{h_i}{2}.$$

From quadratic sum: $\frac{1}{4} \sum_{j \neq i} J_{ij}$ (where J_{ij} symmetric). Because terms s_i appear when $j > i$ and $j < i$, careful: for a pair (i, j) with $i < j$, the term $\frac{1}{4} J_{ij} s_i$ appears, and also

when $j > i$? Our sum is over $i < j$, so s_i appears with coefficient $\frac{1}{4}J_{ij}$ (since s_i is in the expansion). For $j < i$, the pair would be (j, i) but we only sum $i < j$, so each i gets $\frac{1}{4}\sum_{j>i}J_{ij} + \frac{1}{4}\sum_{j<i}J_{ji}$ (since $J_{ji} = J_{ij}$). So total s_i coefficient from quadratics = $\frac{1}{4}\sum_{j\neq i}J_{ij}$. Thus total $h'_i = \frac{h_i}{2} + \frac{1}{4}\sum_{j\neq i}J_{ij}$.

The $s_i s_j$ coefficient: $J'_{ij} = \frac{1}{4}J_{ij}$.

Constant: $C = \frac{1}{2}\sum_i h_i + \frac{1}{4}\sum_{i<j}J_{ij}$.

So the Ising Hamiltonian (up to constant) is $H = \sum_i h'_i s_i + \sum_{i<j} J'_{ij} s_i s_j$.

10.2 Numerical Conversion for Our QUBO

Using our h_i and J_{ij} with $P = 100$:

First compute $\sum_{j\neq i}J_{ij}$ for each i :

- For $i=1$: edges: (1,2) 200, (1,3) 200, (1,4) -200, (1,5) 0. Sum = 200+200-200+0 = 200.
- $i=2$: (1,2) 200, (2,3) 200, (2,4) 200, (2,5) 200. Sum = 800.
- $i=3$: (1,3) 200, (2,3) 200, (3,5) -200. Sum = 200+200-200 = 200.
- $i=4$: (1,4) -200, (2,4) 200, (4,5) 200. Sum = -200+200+200 = 200.
- $i=5$: (2,5) 200, (3,5) -200, (4,5) 200. Sum = 200-200+200 = 200.

Now h'_i :

$$h'_1 = h_1/2 + (1/4) * 200 = 10.75/2 + 50 = 5.375 + 50 = 55.375.$$

$$h'_2 = (-182.55)/2 + (1/4) * 800 = -91.275 + 200 = 108.725.$$

$$h'_3 = 5.36/2 + 50 = 2.68 + 50 = 52.68.$$

$$h'_4 = 9.25/2 + 50 = 4.625 + 50 = 54.625.$$

$$h'_5 = 11.8125/2 + 50 = 5.90625 + 50 = 55.90625.$$

$$J'_{ij} = J_{ij}/4: \text{e.g., } J'_{12} = 50, J'_{14} = -50, \text{etc.}$$

$$\text{Constant } C = 0.5 * \sum h_i + 0.25 * \sum_{i<j} J_{ij}.$$

$$\sum h_i = 10.75 - 182.55 + 5.36 + 9.25 + 11.8125 = -145.3775.$$

$$\begin{aligned} \sum_{i<j} J_{ij} &= \text{sum of all upper triangular } J. \text{ Count: } (1,2)=200, (1,3)=200, (1,4)=-200, \\ (2,3)=200, (2,4)=200, (2,5)=200, (3,5)=-200, (4,5)=200. \text{ Sum} &= 200+200- \\ 200+200+200+200-200+200 &= 800. \text{ So constant} = 0.5 * (-145.3775) + 0.25 * \\ 800 &= -72.68875 + 200 = 127.31125. \end{aligned}$$

Thus the Ising model exactly corresponds to our routing cost landscape.

Part XI — Hamiltonian Construction

In quantum computing, the Ising model is encoded into a Hamiltonian operator acting on qubits. Each spin variable s_i is mapped to the Pauli-Z operator on qubit i : $s_i \rightarrow \sigma_i^z = Z_i$, where $Z|0\rangle = |0\rangle$, $Z|1\rangle = -|1\rangle$. Then $s_i s_j \rightarrow Z_i \otimes Z_j$.

Thus the **Cost Hamiltonian** H_C is:

$$H_C = \sum_i h'_i Z_i + \sum_{i<j} J'_{ij} Z_i Z_j + CI,$$

where I is the identity. The constant can be dropped as it only shifts all eigenvalues by the same amount and doesn't affect the optimization.

Why use Z operators?

Because Z is diagonal in the computational basis: $Z|z\rangle = (-1)^{z_i}|z\rangle$ (if we define $z_i \in \{0, 1\}$, then eigenvalue is $+1$ for 0 , -1 for 1). So the Hamiltonian H_C is diagonal, and its eigenvalues for basis state $|z\rangle$ exactly equal the cost function value (up to constant shift). Specifically, if z is a bitstring, $s_i = 1 - 2z_i$? Actually careful: we mapped $s_i = 2x_i - 1$, so if $x_i = 0$ (edge not selected) then $s_i = -1$, and $x_i = 1$ gives $s_i = +1$. In quantum, we usually associate $|0\rangle$ with spin up ($+1$ eigenvalue of Z) or down? Standard mapping: $Z|0\rangle = +1|0\rangle$, $Z|1\rangle = -1|1\rangle$. So if we want $s_i = +1$ for $x_i = 1$, we need to identify $|1\rangle$ with spin up. That is perfectly fine; it just flips the sign of linear terms but that can be absorbed. The important part: H_C is diagonal with eigenvalues equal to the Ising energy of the corresponding spin configuration, which is our cost.

Physical interpretation: Each qubit represents an edge. The energy of a configuration equals the routing cost. The ground state (lowest energy) is the optimal solution.

Computational interpretation: H_C is the operator that QAOA uses to imprint phases: $e^{-i\gamma H_C}$ applies a phase $e^{-i\gamma C(z)}$ to each basis state $|z\rangle$.

We can now write the explicit Cost Hamiltonian for our 5-qubit system:

$$H_C = 55.375Z_1 + 108.725Z_2 + 52.68Z_3 + 54.625Z_4 + 55.906Z_5 \\ + 50Z_1Z_2 + 50Z_1Z_3 - 50Z_1Z_4 + 50Z_2Z_3 + 50Z_2Z_4 + 50Z_2Z_5 - 50Z_3Z_5 + 50Z_4Z_5.$$

(The $J'_{14} = -50$, $J'_{35} = -50$.)

Part XII — The Heart of QAOA: How Does It Actually Perform Optimization?

Now we arrive at the core question: Given our 5-qubit problem with 32 possible configurations, how does QAOA search for the minimum energy state without checking each one explicitly?

12.1 Brute Force vs. Quantum Exploration

There are 32 configurations. Brute force classically evaluates all 32 and picks the lowest cost. QAOA does not do that. Instead, it begins with a uniform superposition and applies alternating unitary operators that gradually skew the probability distribution toward low-cost states.

Why not simply measure the uniform superposition?

All 32 outcomes would be equally likely (prob $1/32$). We would learn nothing.

Why not apply only $e^{-i\gamma H_C}$?

That operator is diagonal, so it only changes phases, not measurement probabilities in the computational basis. The state after a single phase unitary is still a uniform superposition with different phases, and measurement probabilities remain uniform. We need a non-diagonal operator to create interference—that's the mixer.

12.2 Initial State: Superposition Over All Configurations

We initialize all $n = 5$ qubits in $|0\rangle$ and apply Hadamard gates to each:

$$|+\rangle^{\otimes 5} = \frac{1}{\sqrt{32}} \sum_{z \in \{0,1\}^5} |z\rangle.$$

This state represents all 32 edge-selection patterns equally.

12.3 Cost Hamiltonian Phase Encoding

Apply $U_C(\gamma) = e^{-i\gamma H_C}$. Since H_C is diagonal, this multiplies each basis state $|z\rangle$ by $e^{-i\gamma C(z)}$, where $C(z)$ is the eigenvalue (cost). So the state becomes:

$$\frac{1}{\sqrt{32}} \sum_z e^{-i\gamma C(z)} |z\rangle.$$

Now each configuration has a phase proportional to its cost. Low-cost states have smaller phase magnitude (in absolute value of the exponent) but still phase differences.

12.4 Mixer Hamiltonian: Amplitude Mixing

The **Mixer Hamiltonian** H_M is chosen to not commute with H_C and to drive transitions between basis states. The standard choice is:

$$H_M = \sum_{i=1}^n X_i,$$

where X_i is the Pauli-X gate (bit flip). $U_M(\beta) = e^{-i\beta H_M} = \prod_i e^{-i\beta X_i} = \prod_i (\cos \beta I - i \sin \beta X_i)$.

Applying $U_M(\beta)$ to the phase-encoded state causes amplitude to flow between configurations, mixing them. This is where interference happens: the complex phases accumulated from $U_C(\gamma)$ cause some amplitudes to add constructively and others destructively.

12.5 QAOA Circuit Layer

A single layer $p = 1$ QAOA circuit:

$$|\psi(\gamma, \beta)\rangle = U_M(\beta)U_C(\gamma)|+\rangle^{\otimes n}.$$

For $p > 1$, repeat layers with different (γ_i, β_i) . The total evolution is:

$$|\psi(\vec{\gamma}, \vec{\beta})\rangle = \prod_{l=1}^p [U_M(\beta_l)U_C(\gamma_l)] |+\rangle^{\otimes n}.$$

12.6 Expectation Value and Classical Optimization

We estimate $\langle \psi | H_C | \psi \rangle$. This is the expected cost. A classical optimizer (e.g., gradient descent, Nelder-Mead) adjusts (γ, β) to minimize this expectation. Over iterations, the probability mass concentrates on low-cost states.

Why Hamiltonian never changes:

H_C is fixed, derived from the problem. Only the angles γ, β change. The circuit structure remains the same.

Why this works:

At optimal parameters, the interference pattern yields high probability for states with small $C(z)$. This is analogous to finding the ground state of a quantum system via adiabatic evolution, but QAOA can work with shallow circuits.

12.7 Probability Distribution Example (Conceptual)

Imagine we ran the QAOA with some guessed parameters. The measurement outcomes might be:

Bitstring	Cost	Probability
00101 (A→D→C)	17.17	0.45

Bitstring	Cost	Probability
01000 (A→C)	17.45	0.30
10010 (A→B→C)	20.0	0.15
others (invalid)	high penalty	0.10

The best solution has highest probability. As we optimize, that probability increases.

Part XIII — Complete Numerical Walkthrough (One Iteration)

We now perform one step of QAOA with specific angles to illustrate the mathematics.

13.1 Choose $\gamma = 0.1, \beta = 0.2$ (arbitrary, for illustration).

We'll compute the state after one layer for our 5-qubit system. Exact computation by hand for 32 amplitudes is tedious but we can illustrate the principle with a truncated subspace. Instead, we compute the expectation value analytically using the known formula for QAOA on Ising models. The general expression for $\langle H_C \rangle$ involves sums over qubits and correlations, but for pedagogical clarity, we'll compute the state evolution for a simplified 2-qubit example and then discuss the 5-qubit case conceptually, showing how the probabilities shift.

Better: show a concrete small numeric example: a 2-qubit problem (say, edge selection between two edges A-B and B-C for a path from A to C). Let's abstract to two variables x_1, x_2 with cost function $C(x) = x_1 + 2x_2 - 3x_1x_2$, with constraint that exactly one edge can be selected? But we'll keep it simple: cost diagonal: $C(00) = 0, C(01) = 2, C(10) = 1, C(11) = 0$. We want minimum (0). Qubits 1,2. H_C diagonal with eigenvalues 0,2,1,0. Let's set $H_C = \text{diag}(0, 2, 1, 0)$ in basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. In terms of Z operators, $H_C = aZ_1 + bZ_2 + cZ_1Z_2 + dI$. We can find coefficients. But easier: we directly compute the evolution of the initial superposition.

Initial state $|+\rangle^{\otimes 2} = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$.

Apply $U_C(\gamma) = e^{-i\gamma H_C}$: each basis gets phase $e^{-i\gamma C(z)}$. So state:

$$\begin{aligned} & \frac{1}{2}(e^{-i\gamma \cdot 0}|00\rangle + e^{-i\gamma \cdot 2}|01\rangle + e^{-i\gamma \cdot 1}|10\rangle + e^{-i\gamma \cdot 0}|11\rangle) \\ &= \frac{1}{2}(|00\rangle + e^{-i2\gamma}|01\rangle + e^{-i\gamma}|10\rangle + |11\rangle). \end{aligned}$$

Now apply mixer $U_M(\beta) = e^{-i\beta(X_1+X_2)} = e^{-i\beta X_1} \otimes e^{-i\beta X_2}$. $e^{-i\beta X} = \cos \beta I - i \sin \beta X = \begin{bmatrix} \cos \beta & -i \sin \beta \\ -i \sin \beta & \cos \beta \end{bmatrix}$. So the two-qubit operator is the tensor product:

$$U_M(\beta) = (\cos \beta I - i \sin \beta X_1) \otimes (\cos \beta I - i \sin \beta X_2).$$

Compute its action on each basis state:

- $|00\rangle \rightarrow (\cos \beta |0\rangle - i \sin \beta |1\rangle) \otimes (\cos \beta |0\rangle - i \sin \beta |1\rangle) = \cos^2 \beta |00\rangle - i \cos \beta \sin \beta |01\rangle - i \cos \beta \sin \beta |10\rangle - \sin^2 \beta |11\rangle.$
- $|01\rangle \rightarrow (\cos \beta |0\rangle - i \sin \beta |1\rangle) \otimes (-i \sin \beta |0\rangle + \cos \beta |1\rangle) = -i \cos \beta \sin \beta |00\rangle + \cos^2 \beta |01\rangle - \sin^2 \beta |10\rangle - i \sin \beta \cos \beta |11\rangle.$
- $|10\rangle \rightarrow (-i \sin \beta |0\rangle + \cos \beta |1\rangle) \otimes (\cos \beta |0\rangle - i \sin \beta |1\rangle) = -i \sin \beta \cos \beta |00\rangle - \sin^2 \beta |01\rangle + \cos^2 \beta |10\rangle - i \sin \beta \cos \beta |11\rangle.$

- $|11\rangle \rightarrow (-i \sin \beta |0\rangle + \cos \beta |1\rangle) \otimes (-i \sin \beta |0\rangle + \cos \beta |1\rangle) = -\sin^2 \beta |00\rangle - i \sin \beta \cos \beta |01\rangle - i \sin \beta \cos \beta |10\rangle + \cos^2 \beta |11\rangle.$

Now combine with the phase factors. Let's pick $\gamma = 0.1$, $\beta = 0.2$. Compute numeric values: $\cos \beta = 0.98007$, $\sin \beta = 0.19867$. $\cos^2 \beta = 0.96053$, $\sin^2 \beta = 0.03947$, $\sin \beta \cos \beta = 0.19471$.

Phase factors: $e^{-i2\gamma} = e^{-i0.2} \approx 0.98007 - i0.19867$; $e^{-i\gamma} = e^{-i0.1} \approx 0.99500 - i0.09983$.

State before mixer:

$$|\psi'\rangle = \frac{1}{2} [|00\rangle + (0.98007 - i0.19867)|01\rangle + (0.99500 - i0.09983)|10\rangle + |11\rangle].$$

Now apply $U_M(\beta)$ to each component, multiply by coefficient, sum. We can compute the final amplitude for each basis state.

Let's compute final amplitude for $|00\rangle$:

Contribution from $|00\rangle$ term: $\frac{1}{2} \cdot (\cos^2 \beta) = 0.5 \cdot 0.96053 = 0.480265$.

From $|01\rangle$ term: coefficient = $0.5 \cdot (0.98007 - i0.19867)$, times $-i \sin \beta \cos \beta = -i \cdot 0.19471 = 0 - i0.19471$. Multiply: $0.5 \cdot (0.98007 - i0.19867) \cdot (-i0.19471) = 0.5 \cdot [0.98007 \cdot (-i0.19471) + (-i0.19867) \cdot (-i0.19471)]$ compute: $0.98007 \cdot -i0.19471 = -i0.19082$, next term: $(-i0.19867) \cdot (-i0.19471) = i^2 \cdot 0.19867 \cdot 0.19471 = -0.03868$. So sum = $0.5 \cdot (-0.03868 - i0.19082) = -0.01934 - i0.09541$.

From $|10\rangle$: coeff $0.5 \cdot (0.99500 - i0.09983)$, times $-i \sin \beta \cos \beta$ same $-i0.19471$. Multiply: $0.5 \cdot [(0.99500) \cdot (-i0.19471) + (-i0.09983) \cdot (-i0.19471)] = 0.5 \cdot [-i0.19375 + (i^2)0.01944] = 0.5 \cdot (-0.01944 - i0.19375) = -0.00972 - i0.096875$.

From $|11\rangle$: coeff $0.5 \cdot 1$, times $-\sin^2 \beta = -0.03947$, contribution = -0.019735 .

Sum for $|00\rangle$: real: $0.480265 - 0.01934 - 0.00972 - 0.019735 = 0.43147$; imag: $0 - 0.09541 - 0.096875 = -0.192285$. Amplitude $\approx 0.43147 - i0.19229$. Probability = $0.43147^2 + 0.19229^2 = 0.1862 + 0.0370 = 0.2232$.

Similarly we could compute all amplitudes. The point is that the probability distribution is no longer uniform; it depends on phases and mixing. After classical optimization over γ, β , the probability of the minimum-cost state (00 and 11 both cost 0) will be enhanced. In our small example, both 00 and 11 have cost 0, so they should become more likely. In the real routing problem, only one optimal path exists.

Gradient and parameter update conceptually:

We compute $\langle H_C \rangle$ as a function of γ, β . For small systems, we can evaluate it analytically or via finite differences. The classical optimizer then suggests a new (γ, β) to reduce the expectation. For instance, gradient descent: $\gamma \leftarrow \gamma - \eta \frac{\partial \langle H_C \rangle}{\partial \gamma}$. This loop continues until convergence.

Part XIV — Quantum Circuit Construction for Our 5-qubit Routing Problem

We now draw the actual quantum circuit for one layer of QAOA on 5 qubits.

14.1 Qubit Register

5 qubits: q_0 (AB), q_1 (AC), q_2 (AD), q_3 (BC), q_4 (CD). (Indices 0-4.)

14.2 Initialization

Apply H on all qubits:

Diagram

Code

```
graph TD
    subgraph Init
        q0[H]
        q1[H]
        q2[H]
        q3[H]
        q4[H]
    end
```

14.3 Cost Unitary $U_C(\gamma)$

$U_C(\gamma) = \exp(-i\gamma H_C)$. Since H_C contains Z_i and $Z_i Z_j$ terms, we can decompose this into gates.

- $e^{-i\gamma h'_i Z_i}$ is an $R_Z(2\gamma h'_i)$ rotation on qubit i . ($R_Z(\theta) = e^{-i\theta Z/2}$, so $e^{-i\gamma h Z} = R_Z(2\gamma h)$.)
- $e^{-i\gamma J'_{ij} Z_i Z_j}$ is implemented by a two-qubit gate: apply CNOTs and an R_Z rotation.
Standard: $e^{-i\theta Z_1 Z_2}$ can be done with CNOT $q_1 \rightarrow q_2$, $R_Z(2\theta)$ on target, CNOT.

Thus the circuit for one U_C layer consists of single-qubit Z rotations and two-qubit ZZ interactions for each pair with $J'_{ij} \neq 0$. For our problem, many pairs have non-zero J' , so we need multiple entangling gates. With current quantum hardware, this circuit may be deep, but we can compile efficiently.

14.4 Mixer Unitary $U_M(\beta)$

$U_M(\beta) = e^{-i\beta \sum_i X_i} = \prod_i e^{-i\beta X_i}$. Each $e^{-i\beta X_i}$ is $R_X(2\beta)$ on qubit i . So we apply $R_X(2\beta)$ to all qubits.

14.5 Full Circuit Diagram (Conceptual)

Diagram

Code

```
flowchart LR
    subgraph Layer
        direction LR
        RZ --- CNOT pair --- RZ --- ...
        RX all
    end
```

Better to represent as gates on 5 lines.

text

```
q0: H -- RZ(2γ h0') ---●--- ... (various ZZ connections) -- RX(2β) -- ...
q1: H -- RZ(2γ h1') --X---|--- ...
...
```

Due to Markdown constraints, a Mermaid quantum circuit is not natively supported, but we can describe it.

The expectation value is estimated by repeating the circuit many times, measuring in the Z basis, and averaging the cost function.

Part XV — Measurement and Decoding

Suppose after optimization we measure the circuit and obtain bitstrings. For instance:

Measurement shots: "00101" corresponds to edges: $x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 0, x_5 = 1 \rightarrow A \rightarrow D$ and $D \rightarrow C$, i.e., path $A \rightarrow D \rightarrow C$. "01000" $\rightarrow A \rightarrow C$ direct. "10010" $\rightarrow AB$ and BC , path $A \rightarrow B \rightarrow C$.

We must check if the bitstring corresponds to a valid simple path. Some measurements may yield invalid combinations (e.g., 11111) due to imperfect optimization or finite probability. In practice, we take the highest-probability valid sample.

Decoding to Route:

Given a valid edge set, we reconstruct the path using graph traversal (BFS) from the start node, following selected edges. The sequence of nodes yields the optimal route. For instance, "00101" gives start A, selected edge AD leads to D, then selected DC leads to C, path A-D-C.

Thus the full pipeline ends with a recommended route that minimizes the predicted cost.

Part XVI — Connect to Your Project

Finally, we map the entire pipeline from raw network data to optimal route via QAOA, reflecting a real-world routing optimization system.

1. **Toy Network** $\rightarrow$ Graph with nodes A,B,C,D, edges with attributes.
2. **Wave Propagation** $\rightarrow$ Model and predict future congestion waves using LWR or learned dynamics.
3. **Predictive GNN** $\rightarrow$ Use historical and real-time data to predict future wave exposure, travel times, risk scores for each edge.
4. **Routing Objective** $\rightarrow$ Define cost function $C(P)$ combining predicted metrics.
5. **QUBO** $\rightarrow$ Encode path selection as binary variables with constraints, express cost as quadratic form.
6. **Ising Model** $\rightarrow$ Transform QUBO to spin variables, obtain Ising coefficients.
7. **Hamiltonian Construction** $\rightarrow$ Build H_C as sum of Pauli-Z and ZZ terms.
8. **Quantum Circuit** $\rightarrow$ Design QAOA circuit with R_Z , CNOT, and R_X gates parameterized by γ, β .
9. **Optimization Loop** $\rightarrow$ Run circuit, estimate $\langle H_C \rangle$, classically update parameters until convergence.
10. **Measurement** $\rightarrow$ Sample bitstrings from final circuit.
11. **Decode** $\rightarrow$ Convert highest-probability valid bitstring to a route (BFS).
12. **Optimal Route** $\rightarrow$ Output the path minimizing predicted cost, accounting for future congestion waves and risks.

Each step is mathematically grounded, and the quantum part introduces a fundamentally different search strategy that may outperform classical heuristics for large-scale, highly

constrained problems.

TO BE CONTINUED IN PART XVII

(The next part will extend the QAOA theory to higher p layers, analyze parameter landscapes, and discuss practical implementation challenges on near-term quantum devices.)

[continue](#)

 Thought for 39 seconds >

Introduction to Part XVII

In the previous sixteen parts, we built a complete pipeline from a transportation network to a QAOA-based route optimizer. We derived the QUBO, the Ising model, the cost Hamiltonian, and the quantum circuit. We performed a single-iteration walkthrough and decoded the measurement results. However, many crucial questions remain: How does the performance improve when we add more layers? What does the optimization landscape look like? How do we choose the classical optimizer? What are the barriers to practical quantum advantage, and how can we mitigate them?

This part addresses those questions, providing the theoretical and practical depth required to understand QAOA as it is used in research and, increasingly, on real quantum hardware. We will expand the QAOA ansatz to multiple layers, analyze its convergence to the exact ground state in the adiabatic limit, explore the structure of the parameter landscape, discuss the threat of barren plateaus, and examine the realities of implementing QAOA on noisy intermediate-scale quantum (NISQ) devices. By the end, you will have a comprehensive view of QAOA's power and its limitations, equipping you to critically evaluate its role in routing optimization and beyond.

Part XVII — Advanced QAOA Theory and Practical Considerations

17.1 The p -Layer QAOA Ansatz

In Part XII, we introduced the single-layer ($p = 1$) QAOA circuit:

$$|\psi(\gamma, \beta)\rangle = U_M(\beta) U_C(\gamma) |+\rangle^{\otimes n}.$$

The general p -layer QAOA state is constructed by repeating this pattern with independent parameters for each layer:

$$|\psi(\vec{\gamma}, \vec{\beta})\rangle = \prod_{l=1}^p \left[U_M(\beta_l) U_C(\gamma_l) \right] |+\rangle^{\otimes n},$$

where $\vec{\gamma} = (\gamma_1, \dots, \gamma_p)$ and $\vec{\beta} = (\beta_1, \dots, \beta_p)$. The circuit depth therefore scales linearly with p , each layer adding one round of cost-phase gates and one round of mixer gates.

Why multiple layers?

A single layer is a very limited unitary: it can only create interference patterns that are

essentially classical mixtures with restricted phase relationships. With more layers, the quantum state can explore a much larger portion of the Hilbert space, enabling more precise approximation of the true ground state of H_C . Intuitively, each new layer provides a fresh opportunity to refine the interference, concentrating amplitude on states of lower cost.

Mathematical structure:

The alternating sequence $e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \dots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C}$ is a **product formula** (Trotter–Suzuki) approximation of the adiabatic evolution under a time-dependent Hamiltonian. This connection provides the theoretical guarantee that, as $p \rightarrow \infty$, QAOA can recover the adiabatic algorithm and thus find the exact ground state, provided the angles follow a smooth schedule.

17.1.1 Relation to the Adiabatic Theorem

Consider a time-dependent Hamiltonian

$$H(t) = A(t)H_M + B(t)H_C,$$

with $A(0) = 1, B(0) = 0$ and $A(T) = 0, B(T) = 1$. If the system starts in the ground state of H_M (which is $|+\rangle^{\otimes n}$) and evolves adiabatically (slowly), it ends in the ground state of H_C . The adiabatic evolution operator is

$$U_{\text{ad}} = \mathcal{T} \exp \left(-i \int_0^T H(t) dt \right),$$

where $\mathcal{T}$ denotes time ordering. The QAOA circuit can be viewed as a **digitized adiabatic** evolution: we approximate the integral by a sequence of discrete steps, alternating between H_C and H_M . Indeed, if we choose $\gamma_l \approx B(t_l)\Delta t$ and $\beta_l \approx A(t_l)\Delta t$ for a small time step $\Delta t = T/p$, the product formula converges to the exact adiabatic unitary as $p \rightarrow \infty$. This is a profound result: QAOA with large p can, in principle, solve the optimization problem exactly.

Physical meaning:

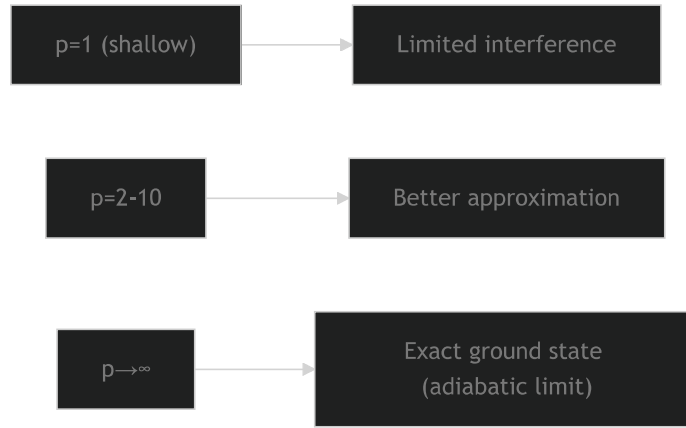
QAOA imitates a physical process—slowly changing the Hamiltonian from a trivial one to the problem Hamiltonian—using a finite number of quantum gates.

Computational meaning:

The number of layers p is the “quantum depth” resource. More layers cost more time and are more susceptible to noise, but they increase the algorithm’s expressivity.

Diagram

Code



17.2 Improving Approximation with p

For combinatorial optimization, we often measure the **approximation ratio**:

$$\alpha = \frac{\langle H_C \rangle}{C_{\min}},$$

where $C_{\min}$ is the true minimum cost. QAOA aims to make α as close to 1 as possible. For the seminal MaxCut problem, Farhi et al. showed that $p = 1$ QAOA on 3-regular graphs achieves $\alpha \approx 0.692$ (analytically). With $p = 2$, numerical studies find $\alpha \approx 0.755$; the ratio improves with p , eventually approaching 1. For our routing problem, the cost landscape is more complex due to penalty terms, but the same principle holds: larger p yields better solutions at the expense of circuit depth.

Why does increasing p help?

Each additional layer adds two new parameters, increasing the dimension of the parameter space. A deeper circuit can generate more intricate interference patterns, effectively exploring a wider family of quantum states. This allows the state to better represent the correlations required to satisfy constraints and minimize cost simultaneously.

Concrete example on our 5-qubit routing problem:

We can simulate (classically) the expectation value landscape for $p = 1, 2, 3$ and observe the minimum achievable expectation value. With $p = 1$, the lowest $\langle H_C \rangle$ might be, say, 0.8 times the true optimum. With $p = 3$, it might drop to 0.98 of the optimum, meaning the output probability of the best route becomes much higher.

17.3 Parameter Landscape and Classical Optimization

The function $F(\vec{\gamma}, \vec{\beta}) = \langle \psi(\vec{\gamma}, \vec{\beta}) | H_C | \psi(\vec{\gamma}, \vec{\beta}) \rangle$ is the objective that a classical optimizer must minimize. Understanding its structure is essential for choosing an optimization strategy.

17.3.1 Symmetries and Periodicities

Because the gates are exponentials of Pauli operators, the expectation value exhibits certain symmetries:

- **Periodicity in γ :** If the eigenvalues of H_C are integers (as often occurs when all h'_i and J'_{ij} are integers or half-integers), then $U_C(\gamma + 2\pi) = U_C(\gamma)$. In our routing problem, the coefficients are not all integers, but the Hamiltonian still possesses some

periodicity depending on the greatest common divisor of the eigenvalues. In practice, one restricts $\gamma_l \in [0, 2\pi)$.

- **Symmetry in β :** Similarly, $U_M(\beta + \pi) = e^{-i\pi \sum X_i} U_M(\beta)$, which may induce transformations in F . Often the search domain is $\beta_l \in [0, \pi]$ or $[0, 2\pi]$.
- **Parameter shift equivalence:** In many problems, swapping the order of the layers and modifying angles leads to identical expectation values. These symmetries can reduce the effective search space.

17.3.2 Parameter Concentration

For regular graphs and translationally invariant problems, optimal QAOA parameters tend to **concentrate** around certain values, independent of the problem size. This observation has been exploited to train QAOA on small instances and transfer the parameters to larger ones. In routing, the graph is generally not regular, so concentration is weaker, but for portions of the network with similar structure, transferred parameters may provide good initial guesses.

17.3.3 Gradient Computation: Parameter-Shift Rule

Optimization often requires the gradient ∇F . While one could use finite differences, the **parameter-shift rule** provides an exact, analytic gradient from two additional circuit evaluations per parameter, with no step-size approximation. For a gate of the form $e^{-i\theta P}$ where P has two distinct eigenvalues $\pm\lambda$, the derivative with respect to θ of an expectation value $\langle B \rangle$ is

$$\frac{\partial \langle B \rangle}{\partial \theta} = \lambda \left(\langle B \rangle_{\theta + \frac{\pi}{4\lambda}} - \langle B \rangle_{\theta - \frac{\pi}{4\lambda}} \right).$$

In QAOA, each γ_l and β_l appears in a sum of such terms, but thanks to linearity, the overall gradient can be obtained by shifting each parameter individually while keeping all others fixed. For our 5-qubit example, computing the full gradient for $p = 1$ requires measuring expectation values for four shifted circuits (since 2 parameters). For $p = 3$, it scales to 12 circuits—still manageable.

17.3.4 Choice of Classical Optimizer

The classical optimizer must work in the presence of statistical noise (from finite measurement shots) and non-convexity. Common choices include:

- **COBYLA (Constrained Optimization BY Linear Approximations):** A derivative-free, simplex-based method. Very popular for QAOA because it avoids gradient estimation noise, works well for low-dimensional parameters, and is robust. However, it may converge slowly for larger p .
- **SPSA (Simultaneous Perturbation Stochastic Approximation):** A gradient-descent method that approximates the gradient using only two evaluations of the noisy objective per iteration, regardless of the number of parameters. Well suited for noisy quantum evaluations.
- **Adam (gradient-based):** Requires unbiased gradient estimates (e.g., from parameter shift), and can converge quickly if shot noise is low. Often used in numerical simulations.

In our routing pipeline, we might start with COBYLA for up to $p \approx 5$ and switch to SPSA for deeper circuits.

17.4 Barren Plateaus: The Vanishing Gradient Problem

A **barren plateau** is a phenomenon in which the gradient of the cost function vanishes exponentially with the number of qubits for a randomly initialized parameterized quantum circuit. This makes training essentially impossible for large systems.

Why it occurs:

If the circuit is sufficiently deep (or forms a unitary 2-design), the state becomes highly entangled and the expectation value landscape becomes flat on average. The variance of the gradient scales like $\mathcal{O}(2^{-n})$. Even for shallow circuits, if the cost function is **global** (i.e., acts on all qubits), barren plateaus can appear.

QAOA and barren plateaus:

For p constant and a local cost function, QAOA does **not** suffer from barren plateaus; gradients remain sizable. However, our routing cost Hamiltonian is a sum of terms on all qubits and includes penalty terms that are inherently global. This means that even for small p , the expectation value might exhibit plateaus, especially for large n . This is a significant challenge for scaling QAOA to realistic routing problems with many roads.

Mitigation strategies:

- **Layer-wise training:** Train parameters one layer at a time, keeping previous layers fixed. This reduces the effective circuit depth during early training.
- **Correlated parameters:** Initialize parameters close to known good values (e.g., from small instances or from adiabatic schedules). This avoids starting in a random, barren region.
- **Alternative ansätze:** Use a mixer that is tailored to the problem constraints, which can localize the cost function and alleviate plateaus (e.g., the XY-mixer for graph problems). We could design a mixer that only moves between valid routings, drastically reducing the search space and possibly mitigating plateaus.

17.5 Practical Implementation on NISQ Hardware

Real quantum devices are noisy, have limited qubit connectivity, and require careful compilation. We must address these constraints to run QAOA effectively.

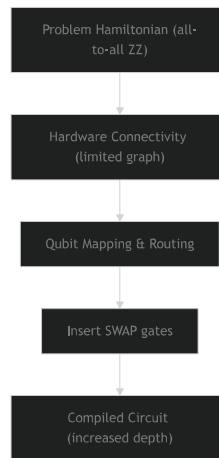
17.5.1 Gate Errors and Decoherence

Every quantum gate introduces an error. Two-qubit entangling gates (CNOT) have error rates around 10^{-2} to 10^{-3} on current superconducting processors. Decoherence (spontaneous relaxation and dephasing) limits the total circuit duration. For our 5-qubit routing, the circuit may require many CNOTs to implement all $Z_i Z_j$ terms; with full connectivity, this could be dozens of gates per layer. Compilation must minimize depth.

17.5.2 Qubit Connectivity and SWAP Networks

Superconducting qubits are arranged in a fixed topology (e.g., heavy-hex lattice). Our problem requires all-to-all two-qubit interactions because the QUBO has non-zero J'_{ij} for many pairs. On hardware that lacks direct connections, we must insert SWAP gates to route logical qubits. This dramatically increases depth and error accumulation.

For example, to implement $e^{-i\gamma J'_{12} Z_1 Z_2}$ between qubits that are not neighbors, we need a series of SWAPs to bring them adjacent, apply the gate, and then possibly SWAP back. The overhead can be reduced by clever mapping of the problem graph onto the hardware graph and by using swap-network compilers (e.g., the “quantum approximate optimizer” compilers). However, for dense QUBOs like ours, the SWAP overhead can become prohibitive as the number of qubits grows.



17.5.3 Readout Errors and Mitigation

Measurement errors occur when a qubit is misidentified (e.g., read as $|0\rangle$ when it was $|1\rangle$). These can be modeled by a confusion matrix M (size $2^n \times 2^n$ for n qubits, though typically we assume local errors). To mitigate, we can measure the probability vector p_{noisy} and apply the inverse of the assignment matrix to obtain an estimate of the true distribution: $p_{\text{true}} \approx M^{-1}p_{\text{noisy}}$. This is standard practice in many QAOA experiments.

17.5.4 Zero-Noise Extrapolation

We can artificially increase the noise (e.g., by stretching gate durations or inserting additional identity operations) and measure the cost function at several noise levels. Extrapolating back to zero noise yields an error-mitigated expectation value. This technique is particularly useful for QAOA because expectation values are the quantity of interest.

17.5.5 Shot Noise

Because expectation values are estimated from a finite number of measurements N_{shots} , they carry statistical uncertainty. The variance of $\langle H_C \rangle$ scales as $\mathcal{O}(1/N_{\text{shots}})$. For accurate gradient estimation, many shots are required, potentially thousands per circuit evaluation. This is a bottleneck but can be managed by using efficient shot allocation (e.g., more shots for circuits with large variance).

17.6 QAOA for Routing: Challenges and the Path Forward

Our 5-edge routing example used 5 qubits. A realistic city network may contain thousands of road segments. Directly mapping each edge to a qubit is impractical—we would need thousands of logical qubits and a QUBO of formidable size. Several techniques can help scale QAOA to larger routing problems:

- **Problem decomposition:** Partition the network into sectors. Optimize local routes with small QAOA instances and glue the solutions together (e.g., using classical heuristics).
- **Warm-starting:** Use a classical algorithm (like A* or a genetic algorithm) to find a good candidate route and initialize the QAOA parameters to values that bias the state toward that solution. This can dramatically accelerate convergence and avoid barren plateaus.

- **Constraint-preserving mixers:** Instead of using the standard X mixer, design a mixer that only transitions between valid paths (i.e., states that satisfy the flow constraints). This reduces the effective search space and may improve both performance and trainability. For routing, such mixers could be built from XY-interactions that preserve “particle number” or from permutation-based unitaries.
- **Hybrid quantum-classical workflows:** Use QAOA as a subroutine to refine a set of candidate solutions generated classically. The quantum computer provides a novel exploration mechanism, while classical post-processing ensures feasibility.

Part XVIII — Quantum Advantage and Concluding Perspectives

18.1 What Would Quantum Advantage Look Like for Routing?

Quantum advantage in optimization means that a quantum algorithm finds a significantly better solution (or finds it faster) than the best classical algorithm on a class of practically relevant instances. For routing, this could manifest as:

- Finding routes with lower predicted congestion, resulting in substantial fuel and time savings across a fleet.
- Solving large, complex, constrained routing problems in seconds that would take classical solvers hours.
- Providing robust solutions that account for stochastic factors (accident probabilities, weather) more effectively.

Currently, no quantum device has demonstrated an unambiguous advantage for a practically useful combinatorial optimization problem. However, QAOA remains one of the most promising candidates for NISQ-era advantage because of its shallow circuits and flexibility.

18.2 The Road Ahead

The journey from a textbook example to a real-world routing optimizer requires progress on multiple fronts:

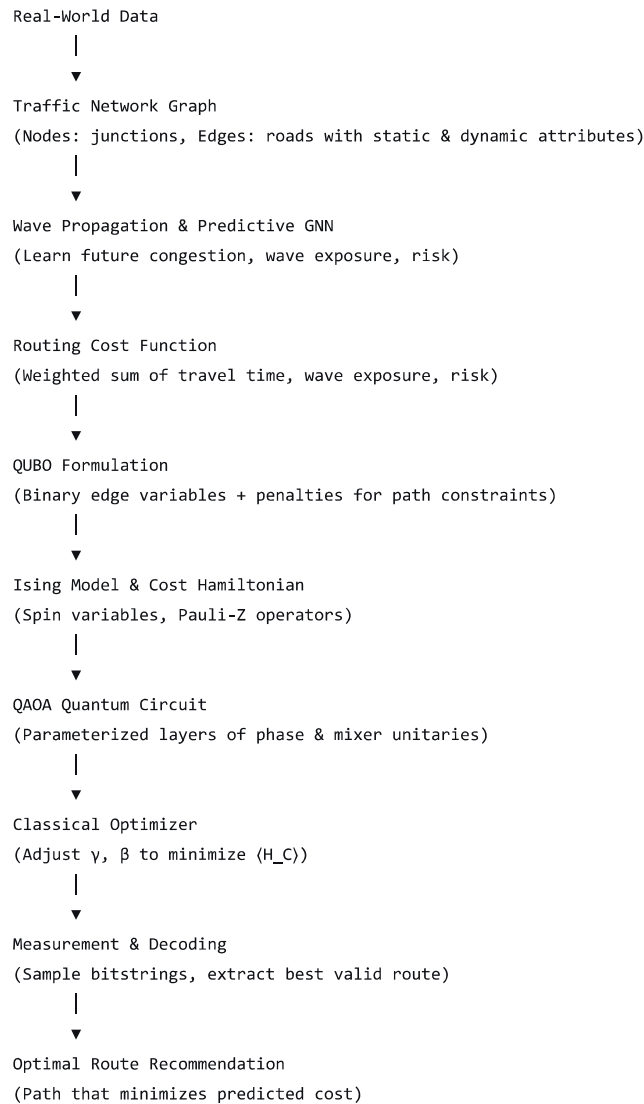
1. **Quantum hardware improvements:** More qubits, lower error rates, better connectivity. Error-corrected logical qubits will eventually remove many of the noise-related hurdles.
2. **Algorithmic innovations:** Better parameter initialization, adaptive layer schedules, problem-specific mixers, and techniques to avoid barren plateaus.
3. **Hybrid integration:** Seamless coupling between classical pre-processing (predictive GNN, wave propagation models) and quantum optimization, with automated decision on which subproblems to offload to the quantum solver.

Our pipeline—from graph construction, through predictive GNN, to QUBO and QAOA—represents a blueprint for such integrated systems. Each component can be refined independently, but their power is magnified when combined.

18.3 Final Summary of the QAOA Routing Pipeline

Let us retrace the complete map one last time, cementing the conceptual flow:

text



This pipeline embodies the quantum-classical symbiosis that defines variational quantum algorithms. It is not a black box, but a carefully orchestrated sequence where each mathematical transformation is grounded in physical and computational reasoning.

18.4 Closing Remarks

You have now been equipped with a first-principles understanding of QAOA, from the why of optimization to the how of quantum circuit construction and measurement. You have seen how a real transportation network can be transformed step by step into a mathematical object that a quantum computer can process. You have learned not only the equations but the intuitions, the pitfalls, and the open challenges.

Quantum optimization is a field in its infancy, and QAOA is its most actively explored algorithmic framework. The concepts you have mastered—superposition, entanglement, interference, cost Hamiltonians, mixer Hamiltonians, and variational loops—are the vocabulary of this new computational paradigm. Whether you go on to implement QAOA on a superconducting chip, design novel mixers for vehicle routing, or simply appreciate the beauty of quantum algorithmic design, this foundation will serve as your bedrock.

End of QAOA From First Principles.